

Event

학습 체크리스트 (1/2)

☐ 브라우저 이벤트와 이벤트 구동 구조 이해

☐ 인라인 속성 방식의 한계 파악

☐ DOM 프로퍼티 이벤트 핸들러의 덮어쓰기 문제 이해

☐ `addEventListener` 로 다중 리스너 등록

학습 체크리스트 (2/2)

☐ `click`, `DOMContentLoaded`의 역할 구분

☐ `submit` 이벤트와 `preventDefault()` 적용

☐ `input`, `change`의 발생 시점 구분

☐ `focus`, `blur`로 입력 상태 감지

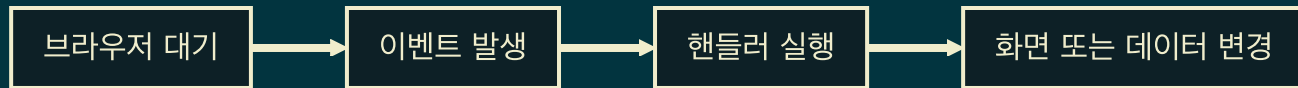
Event란?

이벤트 (Event)

브라우저 안에서 사용자 행동이나 시스템 상태 변화가 발생했음을 알려 주는 신호

- **사용자 행동:** 클릭, 키 입력, 마우스 이동, 폼 전송
- **시스템 변화:** HTML 파싱 완료, 리소스 로딩, 네트워크 상태 변화
- **핵심 관점:** JavaScript가 계속 실행되는 것이 아니라, 사건이 생겼을 때 반응함

이벤트 구동 구조



이벤트 핸들러의 역할

이벤트 핸들러 (Event Handler)

특정 이벤트가 발생했을 때 브라우저가 호출하도록 미리 연결해 둔 함수

- 연결 대상: 버튼, 입력창, 문서, window 같은 객체
- 실행 시점: 이벤트가 실제로 발생한 순간
- 전달 정보: 발생 대상과 입력값은 `event` 객체로 확인

레거시 방식의 위치

- 인라인 속성: HTML 태그 안에 JavaScript를 직접 작성
- DOM 프로퍼티: DOM 객체의 `onclick` 같은 속성에 함수 대입
- 학습 목적: 오래된 코드를 읽기 위해 특성만 이해
- 실무 방향: 새 코드는 `addEventListener` 를 기본으로 사용

인라인 이벤트 핸들러

```
1 <button onclick="alert('클릭됨')">  
2   클릭  
3 </button>
```

인라인 방식의 한계

- 관심사 혼합: HTML 구조와 JavaScript 동작이 한 줄에 섞임
- 수정 어려움: 화면 마크업을 보다가 실행 로직까지 함께 추적해야 함
- 재사용성 저하: 같은 동작을 여러 요소에 깔끔하게 적용하기 어려움
- 유지보수 방향: 구조는 HTML, 동작은 JavaScript로 분리

DOM 프로퍼티 방식

```
1  const button = document.querySelector('.save-btn');  
2  
3  button.onclick = () => {  
4    console.log('저장 버튼 클릭');  
5  };
```

DOM 프로퍼티 방식의 한계

- 단일 슬롯: `onclick`에는 함수 하나만 보관됨
- 덮어쓰기 발생: 새 함수를 대입하면 이전 함수는 사라짐
- 협업 위험: 다른 코드가 같은 프로퍼티를 덮어쓸 수 있음
- 확장 어려움: 독립 기능을 안전하게 누적하기 어렵다

onclick 뉘어쓰기

```
1  const button = document.querySelector('.legacy-btn');
2
3  button.onclick = () => {
4    console.log('A 실행');
5  };
6
7  button.onclick = () => {
8    console.log('B 실행');
9  };
```

addEventListener란?

이벤트 리스너 등록 (addEventListener)

특정 객체에서 특정 이벤트가 발생했을 때 실행할 함수를 등록하는 현대 표준 API

- 첫 번째 인자: 이벤트 이름 문자열
- 두 번째 인자: 이벤트 발생 시 실행할 함수
- 핵심 장점: 같은 이벤트에 여러 리스너를 누적 가능

addEventListener 기본 문법

```
1  const button = document.querySelector('.save-btn');  
2  
3  button.addEventListener('click', () => {  
4    console.log('저장 버튼 클릭');  
5  });
```

다중 리스너 등록

```
1  const button = document.querySelector('.save-btn');
2
3  button.addEventListener('click', () => {
4    console.log('저장 요청');
5  });
6
7  button.addEventListener('click', () => {
8    console.log('분석 로그 기록');
9  });
```

event 객체

이벤트 객체 (Event Object)

이벤트가 발생했을 때 브라우저가 핸들러에 전달하는 상세 정보 객체

- 발생 대상: `event.target`
- 기본 동작 제어: `event.preventDefault()`
- 입력값 확인: 입력 요소에서는 `event.target.value`

클릭 대상 확인

```
1  const button = document.querySelector('.save-btn');  
2  
3  button.addEventListener('click', (event) => {  
4    console.log(event.target);  
5  });
```

click 이벤트

- 발생 시점: 요소를 마우스로 클릭하거나 터치로 선택할 때
- 주요 대상: 버튼, 링크, 메뉴, 카드형 UI
- 대표 용도: 저장, 삭제, 열기, 선택 같은 명령 실행
- 주의점: 입력 변화 감지는 `click` 보다 `input`, `change` 가 적합

DOMContentLoaded 이벤트

- 발생 시점: HTML 파싱이 끝나 DOM 트리가 준비된 순간
- 대기하지 않음: 이미지나 스타일시트 전체 로딩까지 기다리지 않음
- 사용 목적: 외부 스크립트에서 DOM 요소를 안전하게 찾기
- 비교 기준: DOM 탐색 준비는 `DOMContentLoaded`, 전체 리소스 완료는 `load`

DOM 준비 후 버튼 연결

```
1 document.addEventListener('DOMContentLoaded', () => {  
2   const button = document.querySelector('#submit-btn');  
3  
4   button.addEventListener('click', () => {  
5     console.log('클릭 이벤트 접수');  
6   });  
7 });
```

removeEventListener의 조건

- 용도: 더 이상 필요 없는 리스너를 제거
- 핵심 조건: 등록할 때 쓴 함수 참조가 필요
- 익명 함수 한계: 같은 모양으로 다시 써도 같은 함수가 아님
- 사용 상황: 일회성 UI, 모달 정리, 컴포넌트 제거

리스너 제거를 위한 함수 분리

```
1  const button = document.querySelector('.close-btn');
2
3  function handleClose() {
4    console.log('닫기 처리');
5  }
6
7  button.addEventListener('click', handleClose);
8  button.removeEventListener('click', handleClose);
```

form submit 이벤트

폼 전송 이벤트 (submit)

사용자가 폼을 제출하려고 할 때 `<form>` 요소에서 발생하는 이벤트

- 발생 조건: 제출 버튼 클릭 또는 입력창에서 Enter
- 기본 동작: `action` 주소로 전송하고 페이지를 새로고침
- 현대 앱 제어: JavaScript 처리 전에는 기본 동작을 막는 경우가 많음

폼 기본 새로고침 차단

```
1  const form = document.querySelector('#user-form');  
2  
3  form.addEventListener('submit', (event) => {  
4    event.preventDefault();  
5  
6    console.log('새로고침 없이 처리');  
7  });
```

input 이벤트

- 발생 시점: 글자가 입력되거나 삭제될 때마다 즉시 발생
- 주요 대상: 텍스트 입력창, 검색창, textarea
- 대표 용도: 실시간 검색, 글자 수 표시, 즉시 유효성 검사
- 값 확인: 현재 입력값은 `event.target.value`

실시간 입력값 읽기

```
1  const searchInput = document.querySelector('#search');  
2  
3  searchInput.addEventListener('input', (event) => {  
4    const keyword = event.target.value;  
5  
6    console.log('검색어:', keyword);  
7  });
```

change 이벤트

- 발생 시점: 값 변경이 확정되었을 때
- 텍스트 입력: 값을 바꾼 뒤 포커스를 잃으면 발생
- 선택 요소: select, checkbox, radio는 선택 변경 시 적합
- 비교 기준: 타이핑 즉시는 `input`, 확정된 변경은 `change`

선택값 변경 감지

```
1  const roleSelect = document.querySelector('#role-select');
2
3  roleSelect.addEventListener('change', (event) => {
4    const selectedRole = event.target.value;
5
6    console.log('선택된 권한:', selectedRole);
7  });
```

focus와 blur

- `focus`: 입력 요소에 커서가 들어왔을 때 발생
- `blur`: 입력 요소가 포커스를 잃었을 때 발생
- **focus 활용**: 입력 가이드, 강조 스타일, 보조 메시지 표시
- **blur 활용**: 입력 완료 후 이메일 형식이나 비밀번호 조건 검사

포커스 진입 안내

```
1  const emailInput = document.querySelector('#email');  
2  
3  emailInput.addEventListener('focus', () => {  
4    console.log('이메일 형식으로 입력해 주세요.');
```

포커스 이탈 검증

```
1  const emailInput = document.querySelector('#email');
2
3  emailInput.addEventListener('blur', (event) => {
4    const email = event.target.value;
5
6    if (!email.includes('@')) {
7      console.warn('이메일 형식이 아닙니다. ');
8    }
9  });
```

입력 이벤트 선택 기준

상황	적합한 이벤트
버튼 클릭으로 명령 실행	click
DOM 준비 후 요소 탐색	DOMContentLoaded
폼 제출을 JS로 처리	submit
타이핑 중 즉시 반응	input
선택 완료 후 반응	change
입력 시작·종료 감지	focus, blur